Bloom Filter-Based MPSI

Weekly Progress Meeting 19 Feb 2026

Andra Alăzăroaie

Supervisor: Lilika Markatou

Daily Supervisor: Tjitske Koster

19-02-2026

Bibliography

[1] Jelle Vos, Jorrit van Assen, Tjitske Koster, Evangelia Anna Markatou, and Zekeriya Erkin. On the insecurity of bloom filter-based private set intersections. Cryptology ePrint Archive, 2024.

[3] Ou Ruan, Changwang Yan, Jing Zhou, and Chaohao Ai. A practical multiparty private set intersection protocol based on bloom filters for unbalanced scenarios. Applied Sciences, 13(24):13215, 2023.

[6] Vladimir Kolesnikov, Naor Matania, Benny Pinkas, Mike Rosulek, and Ni Trieu. Practical multi-party private set intersection from symmetric-key techniques.In Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, pages 1257–1272, 2017.

[7] Alireza Kavousi, Javad Mohajeri, and Mahmoud Salmasizadeh. Efficient scalable multi-party private set intersection using oblivious prf. In International Workshop on Security and Trust Management, pages 81–99. Springer, 2021.

[8] Florian Kerschbaum. Outsourced private set intersection using homomorphic encryption. In Proceedings of the 7th ACM Symposium on Information, Computer and Communications Security, pages 85–86, 2012.

[9] Casacuberta, Sílvia, Julia Hesse, and Anja Lehmann. "SoK: oblivious pseudorandom functions." In 2022 IEEE 7th European Symposium on Security and Privacy (EuroS&P), pp. 625-646. IEEE, 2022.

[10] Dong, Changyu, Liqun Chen, and Zikai Wen. "When private set intersection meets big data: an efficient and scalable protocol." In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, pp. 789-800. 2013.

Progress

From the plan last week:

- Implement the OPRF mitigation
- Start the Garbled BF mitigation ([6] and [7] use GBFs to implement OPRF)
- Read about the Authorized MPSI mitigation [8]

Progress

OPRFs:

- Use this OPRF

```
// Server S initiates a request:
Request(M) → t, B
    t ←$ Z_q
    B ← {a^t}_{a∈M}
    return t, B

// Client C applies PRF:
Eval(B, k) → C
    return {b^k}_{b∈B}

// Server S recovers elements:
Recover(C, t) → D
    return {c^(1/t)}_{c∈C}
```

Figure 8: OPRF Protocol

Progress

OPRFs:

Clients generate a key k_{OPRF}

- Two rounds: one to compute the key, one to distribute it
- Take clients A, B, C:
  - A computes g^a, sends to B
  - B computes g^{ab}, sends to C
  - C computes $g^{abc} = k_{OPRF}$, sends to A, A sends to B, B sends to C

Progress

OPRFs:

Pre-processing stage:

- Original protocol: each client hashes their element using the k hash functions to get the indices in the BF
- Mitigation: each client computes the indices by using their k_{OPRF}
  - For element x: $H_1(H(x)^{k_{OPRF}}\ mod\ p), H_2(H(x)^{k_{OPRF}} mod\ p), \dots, H_k(H(x)^{k_{OPRF}} mod\ p)$
  - $\gcd(k_{OPRF}, \mathrm{p} - 1) = 1$, so $H(x)^{k_{OPRF}}\ mod\ p$ will be unique for $H(x)$
  - The hash function $H(x)$ needs to generate uniformly random elements, something like SHA256 should work?
  - $H_1, H_2, \dots, H_k$ are the previous hash functions mapping elements to bins

Progress

OPRFs:

Online stage:

- Original protocol: server computes the k hashes for each of its elements to get indices and collect the encrypted values at those indices from the clients' encrypted BFs
- Mitigation: server has to engage in an OPRF protocol in order to get the indices
  - One OPRF is sufficient since all clients have the same k_{OPRF}

Next Week

- Run the experiments on the OPRF mitigation
- Implement the Garbled BF mitigation
- Read more and do some thinking on the Authorized MPSI mitigation

Thank you!

19-02-2026